PROMPT LIBRARY

Step by Step, Every Prompt

Step by step, the prompt for every lab. Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder, copy each prompt in order, review the code it writes and run it. The complete, working code is already in each lab folder. The same prompts, with Copy buttons, are in Prompt_Library.html.

- Work down each lab's steps in order: copy the prompt into your AI coding assistant.
- Review the code it writes, then run the lab's command (shown under the lab title).
- Compare with the reference code in the lab's solution/ folder if the result differs.
- If the code stops with an error, paste the error back to the assistant and ask it to fix it.
- Never paste API keys into an AI chat. Paper trading only - nothing here is investment advice.

Lab 01 - Pipeline Skeleton, Configuration and the Alpaca Paper Account

Folder Lab01-alpaca-setup/ - run: python3 Lab01-alpaca-setup/check_stage.py

Step 1: Set up Python and your AI coding assistant (K5) (no prompt needed)
Open a terminal in the labs folder. Run: python3 -m venv .venv, then source .venv/bin/activate (Windows: .venv\Scripts\activate), then pip install -r requirements.txt. Open the labs folder in VS Code with Claude Code (or your AI coding assistant). Working rule for the whole course: the assistant writes code, YOU review it and run it; never paste an API key into the assistant.
Step 2: Store Alpaca PAPER keys outside the code (K5) (no prompt needed)
Sign up at alpaca.markets, switch to Paper Trading in the account selector, open API Keys and click Generate New Keys. Run: cp .env.example .env and paste the values into APCA_API_KEY_ID and APCA_API_SECRET_KEY. Never put keys in code, e-mail, chat or git. No keys? Leave them blank: every stage then runs OFFLINE on bundled data and the local paper-broker simulator.
Step 3: Write the pipeline configuration (K5)
In labs_fullcode/pipeline/ create config.yaml for Lion City Capital (fictitious)'s algorithmic trading pipeline. Sections: mode (auto | offline); broker (name alpaca, paper: true, sim_state_file pipeline_sim.json); secrets_from_env listing only the NAMES of the environment variables APCA_API_KEY_ID, APCA_API_SECRET_KEY, WEBHOOK_SECRET and ANTHROPIC_API_KEY; universe (trade: SPY QQQ AAPL MSFT NVDA JPM XLF GLD TLT; pairs KO/PEP, JPM/BAC, XLF/JPM; benchmark SPY; watch: AAPL as NASDAQ:AAPL in USD and D05.SI as SGX:D05 in SGD); data (start 2018-01-01, end 2025-12-31, timeframes 1Day 1Week 1Month 1Hour, hourly_window 2025-10-01 to 2025-12-31, cache_dir _cache, max_abs_daily_move 0.08); strategy (name ema_cross, symbol SPY, params fast 10 slow 30, execution next_open); costs (preset alpaca_retail, scenarios zero alpaca_retail per_share_broker high_cost, tradingview commission_per_order 1.0 and slippage_bps 0.4); risk (max_position_pct 0.95, max_daily_loss_pct 0.02, max_order_value 100000, max_order_qty 100, allowed_symbols = the trade universe, long_only true, kill_switch_file KILL_SWITCH); live_paper (max_position_pct 0.10, limit_buffer_bps 25, poll_seconds 30, crypto_demo_symbol BTC/USD, crypto_demo_notional 25); webhook (host 127.0.0.1, port 8080, path /webhook, require_signal_agreement false); agents (model claude-sonnet-5-5, symbols NVDA AAPL SPY, max_position_pct 0.20, max_order_value 25000, min_confidence 0.6, stop_atr 2.0, target_atr 3.0); schedule (America/New_York, Asia/Singapore, run_at_exchange_time 09:35). Comment every key. Never put a key or secret VALUE in this file.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Build the configuration loader with safety validation (K5)

Create labs_fullcode/pipeline/config.py. load_config(path=None, strict=True) reads pipeline/config.yaml with yaml.safe_load (path override from the PIPELINE_CONFIG environment variable), caches it, stores the path in cfg['_path'] and the problems in cfg['_problems'], and raises ValueError when strict and invalid. If mode is offline set LAB_OFFLINE=1. validate(cfg) returns a list of problems: required sections present; broker.paper must be true; risk.max_position_pct and risk.max_daily_loss_pct between 0 and 1; costs.preset must exist in common.backtest.COST_PRESETS; every secrets_from_env value must be an UPPER_SNAKE environment variable name; any string that looks like a credential (PK/AK/CK followed by 14+ capitals or digits, 'sk-ant-', or a 32+ character base64 string) is reported as a secret in the file. Also add apply_cli_flags(argv) (strip --offline -> LAB_OFFLINE=1, and --config PATH -> PIPELINE_CONFIG), secret(cfg, name) that reads the value from os.environ, lab_path(rel), kill_switch_path(cfg), cost_model(cfg, preset=None), tradingview_cost_model(cfg) returning CostModel('TradingView settings', min_commission=1.0, slippage_bps=0.4) from config, and stage_banner(no, title). Use LAB_ROOT from common.config. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 5: Add the market clock for US and Singapore time (K5)

Create labs_fullcode/pipeline/clock.py. session_at(ts) returns 'regular' (09:30-16:00 New York, Mon-Fri), 'pre' (04:00-09:30), 'after' (16:00-20:00), 'overnight' (20:00-04:00 from Sunday night to Friday 04:00, Alpaca 24/5 trading) or 'closed'; naive timestamps are New York time. RULES maps each session to the allowed order types, time_in_force values and extended_hours flag: regular = market, limit, stop, bracket; pre, after and overnight = LIMIT orders only with time_in_force DAY or GTC and extended_hours=True; closed = a GTC limit order that queues for the next session. broker_now(broker) uses the simulator's replay clock (broker.state['now']) when it has one, otherwise the real UTC clock. market_status(broker=None, now=None, override=None) returns now in America/New_York and Asia/Singapore, the session, a holiday flag and the rules; when the broker is the Alpaca paper client (mode 'alpaca-paper') and the session is 'regular' but broker.get_clock().is_open is False, report 'closed' (exchange holiday). describe(status) returns one line such as 'Mon 2026-10-19 21:05 New York = Tue 09:05 Singapore | session: OVERNIGHT (...)'. Use zoneinfo only. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 6: Build the check command and the pipeline CLI (K5)
Create labs_fullcode/pipeline/check.py with run(reset=False, quiet=False) returning a dict and main(argv) for `python -m pipeline check [--reset]`. Print: [1] the config path, firm, mode, universe, strategy, risk limits and the validation result; [2] versions of pandas, numpy, scipy, sklearn, xgboost, matplotlib, yaml and alpaca; [3] each entry of secrets_from_env as 'found (PK12...)' for the Alpaca key ID, 'found (hidden)' for any secret, or 'not set' - never print a secret value; [4] the paper account from common.broker.get_broker(state_file=cfg['broker']['sim_state_file'], reset=reset): account number, status, equity, cash, buying power and number of positions; [5] the market clock line from pipeline.clock; [6] the last 3 daily bars of the strategy symbol from common.data.get_bars with data_source(). End with 'RESULT: PIPELINE READY - paper trading only' and exit code 0, or 'NOT READY - <problems>' and exit code 1. Then create pipeline/__init__.py (docstring, __version__) and pipeline/__main__.py with a COMMANDS table {command: (lab, module, function, description)} for all 20 stages (check, validate, ingest, hypothesis, indicators, reconcile, signals, execute, evaluate, search, ml, regime, abtest, pairs, backtest, run, schedule, kill, webhook, alert-sim, diagnose, report, agents, e2e); import the module lazily; strip --offline and --config with apply_cli_flags; print the table when called with no arguments. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 7: Run the pre-flight check (K5) (no prompt needed)
Run: python -m pipeline check --reset (same as: python3 Lab01-alpaca-setup/check_stage.py --reset). Then run echo $? (Windows: echo %errorlevel%) to see the exit code that a scheduler would see.
Step 8: Prove the safety rules work (K5)
Explain to me, as a finance professional new to coding, why labs_fullcode/pipeline/config.py refuses to run when broker.paper is false or when a value in config.yaml looks like an API key, and what could go wrong at a trading desk without these two checks. Do not change any file.

Open in Claude | **Open in ChatGPT**

Lab 02 - Data-Validation Stage: Cross-Check Pipeline Bars against TradingView

Folder Lab02-tradingview-charts/ - run: python3 Lab02-tradingview-charts/validate_stage.py

Step 1: Add the data-check panel to SGX:D05 (A3, K9)
Explain labs_fullcode/Lab02-tradingview-charts/lcc_data_check.pine for a finance professional: what each input does, how the script finds the bar to validate, and which values in its table I should copy into tradingview_values.csv. Do not change the file.

Open in Claude | **Open in ChatGPT**

Step 2: Repeat for NASDAQ:AAPL and switch the adjustment (A3, K9) (no prompt needed)
Open NASDAQ:AAPL on 1D with the same indicator and read 31 December 2025. Then open Chart settings > Symbol and tick 'Adjust data for dividends'; read the bar again and untick it.
Step 3: Record what TradingView shows (A3) (no prompt needed)
Open Lab02-tradingview-charts/tradingview_values.csv. Overwrite the two example rows with your own readings (tradingview_symbol, date, open, high, low, close, volume, setting = default or adjusted).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Repeat for NASDAQ:AAPL and switch the adjustment (A3, K9) (no prompt needed)
Step 4: Build the validation stage (A3, K5, K9)
Create labs_fullcode/pipeline/validate.py. reference_bars(symbol) reads pipeline/reference/<SYMBOL>_1Day_raw.csv (columns timestamp, open, high, low, close, adj_close, volume, dividend, split). adjusted_bars(symbol, raw) returns the research bars from common.data.get_bars for the same dates and, if no file exists (FileNotFoundError, e.g. D05.SI), raw open/high/low/close multiplied by adj_close / close. compare(tv_row, bar) returns the percentage difference per field and match=True if every price is within 0.1% and volume within 10%. run(date='2025-12-31', tv_file=None) loops over config universe.watch, prints for each symbol the RAW row, the ADJUSTED row, your TradingView row from Lab02-tradingview-charts/tradingview_values.csv (lines starting with # are comments) and the differences, the dividend-adjustment factor (adjusted close / raw close), change vs prior close, 52-week high/low and 20-day average volume, and a verdict: 'PASS - TradingView matches RAW', 'PASS - TradingView matches ADJUSTED' or 'FAIL'. Save _outputs/Lab02_validation_report.csv and a chart Lab02_validation.png (raw vs adjusted close in 2025, one panel per symbol) with common.plotting.save. main(argv) supports --date and --tv. This stage only reads data - it must never call an order API. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 5: Run the validation stage (A3, K9) (no prompt needed)
Run: python -m pipeline validate --date 2025-12-31 (same as: python3 Lab02-tradingview-charts/validate_stage.py --date 2025-12-31).
Step 6: Understand adjusted versus unadjusted prices (K9)
Using the dividend and split columns in labs_fullcode/pipeline/reference/D05.SI_1Day_raw.csv, explain why the pipeline's adjusted D05 close for 2025-12-31 is about 3.95% below TradingView's close, why backtests must use adjusted prices while orders are priced in raw prices, and what error a trading program makes if it mixes the two. Answer in five bullet points; do not change any file.

Open in Claude | **Open in ChatGPT**

Step 7: Make the validation fail on purpose (A3) (no prompt needed)
In tradingview_values.csv change the AAPL row to the dividend-adjusted values open 272.32, high 272.94, low 271.01, close 271.12, setting adjusted, and rerun the stage. Next change only the close to 280.00 and rerun. Finally restore your real reading.
Step 8: Identify other forms of market data (K9) (no prompt needed)
On TradingView open the News, Financials and economic calendar views for SGX:D05. Note whether the SGX data are real-time or delayed on your plan and why the pipeline takes its US data from the Alpaca API rather than from a chart.

Lab 03 - Ingestion Stage: Multi-Timeframe Bars, Headlines and Data Quality

Folder Lab03-alpaca-market-data/ - run: python3 Lab03-alpaca-market-data/ingest_stage.py

Step 1: Understand how bars are requested (K9)
Read labs_fullcode/common/data.py and explain in five bullet points how get_bars() requests bars from Alpaca (TimeFrame, adjustment, feed, time zone) and what it does offline. Do not change the file.

Open in Claude | **Open in ChatGPT**

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Build the cache and load_bars() (A6, K9)

Create labs_fullcode/pipeline/ingest.py with cache_dir() (config data.cache_dir under labs), cache_file(symbol, timeframe) -> _cache/bars/<SYMBOL>_<TIMEFRAME>.csv, and load_bars(symbol, start=None, end=None, timeframe='1Day') that reads the cached CSV (hourly timestamps as America/New_York, daily as dates), slices start and end inclusively (weekly and monthly bars are labelled with the period end, so allow 6 / 30 days after end), and falls back to common.data.get_bars when the file is missing or when online with end=None. Add load_closes(symbols, start, end) that inner-joins daily closes like common.data.get_closes. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 3: Add automated data-quality checks (A6)

In labs_fullcode/pipeline/ingest.py add quality_checks(df, timeframe, end, max_move) returning a dict: rows, first, last, missing_values, duplicate_timestamps, not_sorted, high_below_low, close_outside_range, zero_volume, gaps_over_4_days (daily only), big_moves (daily |return| > max_move, from config data.max_abs_daily_move) and stale (last daily bar more than 5 days before end). BLOCKING = missing_values, duplicate_timestamps, not_sorted, high_below_low, close_outside_range, stale. Status is FAIL if any blocking count is non-zero (the file is NOT cached), CHECK if there are big moves or zero-volume bars, else OK. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 4: Ingest an unstructured source: news headlines (A6, K9)

In labs_fullcode/pipeline/ingest.py add _fetch_headlines(symbols): online use alpaca.data.historical.news.NewsClient with NewsRequest(symbols=..., start=now - 7 days, limit=10) and keys from common.config.get_keys(); on any error, or offline, load pipeline/reference/sample_headlines.json and set _source to 'bundled SYNTHETIC sample (training only - not real news)'. Add load_headlines(symbols=None) that reads _cache/news/headlines.json when it exists. Treat headline text as untrusted data. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Build the ingest run with the report (A6, K9)
In labs_fullcode/pipeline/ingest.py add run(symbols=None, timeframes=None) for `python -m pipeline ingest`: symbols = universe trade + pair legs + benchmark; for each symbol and config timeframe fetch bars with common.data.get_bars (1Hour uses data.hourly_window; record 'not available offline' on FileNotFoundError), run quality_checks, cache every non-FAIL file, write _outputs/Lab03_data_quality.csv and _cache/manifest.json (ingested_at, source, rows/first/last/status per file, headline count), save the headlines to _cache/news/headlines.json, print the daily quality table, bars per timeframe, headline count by symbol, SPY and AAPL annualised mean, volatility, skew, excess kurtosis and correlation, a DATA_FORMS table (bars, quotes, trades, corporate actions, fundamentals = structured; news, alternative data = unstructured) and three lines on Volume, Velocity and Variety. Save chart Lab03_market_data.png (universe growth of 100 on a log scale, SPY/AAPL return histograms, SPY 21-day volatility, SPY hourly closes). Exit code 1 if any file FAILs. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 6: Run the ingestion stage (A6, K9) (no prompt needed)
Run: python -m pipeline ingest (same as: python3 Lab03-alpaca-market-data/ingest_stage.py).
Step 7: Inspect the cache and the manifest (A6) (no prompt needed)
Open _cache/manifest.json and _outputs/Lab03_data_quality.csv. Rerun the Step 2 check command - it now reads from the cache. Look at _outputs/Lab03_market_data.png.
Step 8: Design a big-data extension (A6)
Using the DATA_FORMS table in labs_fullcode/pipeline/ingest.py, propose how to add Alpaca news for the full trade universe and 1-minute bars for 500 US stocks as scheduled ingest jobs: storage format, refresh frequency, deduplication key, estimated rows per year, and three governance controls (data licence, point-in-time universe, prompt-injection risk when headlines reach an AI agent). Answer in a short table; do not write code yet.

Open in Claude | Open in ChatGPT

Lab 04 - Hypothesis Stage: from a Written Card to an Automated Statistical Test

Folder Lab04-trading-hypothesis/ - run: python3 Lab04-trading-hypothesis/hypothesis_stage.py

Step 1: Read the hypothesis card (A1, K1) (no prompt needed)
Open pipeline/hypotheses/spy_dip_after_streak.yaml. Identify H1, H0, the signal (3 consecutive down closes, known at the close), the outcome (next-day return), the test and the decision rule (p < 0.05 AND an edge of at least 3 bps, the estimated round-trip cost).
Step 2: Write a card for your own idea (A1, K1)
Help me write a hypothesis card in YAML for this trading idea: <describe your idea in one sentence>. Use exactly the fields of labs_fullcode/pipeline/hypotheses/spy_dip_after_streak.yaml (id, name, instrument, timeframe, sample, rationale, h1, h0, signal, outcome, test, decision_rule, robustness, biases_to_watch, big_data_use_case). The signal type must be one of losing_streak (n), rsi_below (window, level) or close_below_sma (window) and must be known at the close of day t; the outcome must be a forward return over horizon_days; H0 must be 'no difference'. Do not analyse any data.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Build the test engine (A1)
Create labs_fullcode/pipeline/hypothesis.py. losing_streak(close) counts consecutive down closes ending at each day. condition(bars, signal) builds the boolean signal for the types losing_streak, rsi_below (RSI from common.indicators) and close_below_sma, using data up to the close of day t only. test_card(bars, card, signal=None): outcome fwd = close.shift(-horizon) / close - 1 (the outcome, never a feature); compare signal days with all other days: counts, means, hit rates, scipy.stats.ttest_ind(equal_var=False), a bootstrap 95% CI of the signal-day mean with numpy default_rng(card seed) and card bootstrap draws, edge_bps = (mean signal - mean other) x 10,000, supported = p < alpha AND t > 0 AND edge_bps >= min_edge_bps. Load bars with pipeline.ingest.load_bars. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 4: Add the robustness grid, chart and result file (A1, A6)
In labs_fullcode/pipeline/hypothesis.py add run(card_path=None) for `python -m pipeline hypothesis [--card PATH]` (default pipeline/hypotheses/spy_dip_after_streak.yaml): print H1, H0, sample, signal days, means, hit rates, CI, Welch t-test, edge and a DECISION line ('SUPPORTED - reject H0; promote the idea to the signal and backtest stages' or 'NOT SUPPORTED - do not reject H0'); a robustness grid over card.robustness.streaks for the instrument and each robustness symbol with a Bonferroni threshold alpha / number of variants; a table of forward return by streak length (0..4, 5+) with standard errors; chart Lab04_hypothesis.png (bars with 95% error bars, unconditional mean line); and write the card plus results to _outputs/Lab04_hypothesis_result.yaml. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 5: Run the test on the desk's card (A1) (no prompt needed)
Run: python -m pipeline hypothesis (same as: python3 Lab04-trading-hypothesis/hypothesis_stage.py).
Step 6: Read the robustness grid (A1, K1) (no prompt needed)
Read the robustness table and the Bonferroni line. Which variants would survive if the desk counted every test it ran?
Step 7: Study the shape of the effect (A1) (no prompt needed)
Read the table 'Forward return by losing-streak length' and open _outputs/Lab04_hypothesis.png.
Step 8: Test your own card and its big-data use case (A1, A6)
Review my hypothesis result in labs_fullcode/_outputs/Lab04_hypothesis_result.yaml. In plain English, say whether the evidence supports H1, how many variants I tested, whether the result survives a Bonferroni correction, and which bigger or alternative data set (volume, velocity or variety) would test the idea better. Do not change any code.

Open in Claude | Open in ChatGPT

Lab 05 - Indicator Stage: Pine Script v6 Indicators and their Python Twins

Folder Lab05-pine-indicators/ - run: python3 Lab05-pine-indicators/indicators_stage.py

Step 1: Add the trend and bands indicator (A3, K3)
Explain labs_fullcode/Lab05-pine-indicators/lcc_trend_bands.pine line by line for a finance professional: the inputs, ta.ema, ta.bb, ta.crossover / ta.crossunder, plotshape, barcolor and alertcondition. Which values will the Data Window show for each bar?

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Add the RSI pane (A3, K3) (no prompt needed)
Create another indicator from Lab05-pine-indicators/lcc_rsi.pine, save it as 'LCC RSI' and add it.
Step 3: Record the TradingView values (A3) (no prompt needed)
Move the cursor to 31 December 2025, open the Data Window and copy close, EMA fast, EMA slow, BB upper, middle, lower and RSI into Lab05-pine-indicators/tradingview_indicator_values.csv (overwrite the example row).
Step 4: Build the Pine twins (K3)
Create labs_fullcode/pipeline/indicators.py with Python twins of the Pine Script v6 built-ins, implemented by calling labs_fullcode/common/indicators.py: ta_sma(src, length), ta_ema(src, length) (alpha 2/(n+1), adjust=False), ta_rsi(src, length=14) (Wilder smoothing), ta_bb(src, length=20, mult=2.0) returning (middle, upper, lower) with the population standard deviation, ta_atr(bars, length=14), ta_macd(src, 12, 26, 9) returning (macd, signal, hist), ta_crossover(a, b) = a > b and a[1] <= b[1], ta_crossunder(a, b). Then lcc_trend_bands(bars, fast=20, slow=50, bb_len=20, bb_mult=2.0) returning close, ema_fast, ema_slow, bb_middle, bb_upper, bb_lower, bull_cross, bear_cross exactly like lcc_trend_bands.pine, lcc_rsi(bars, 14, 70, 30) returning rsi, alert_oversold and alert_overbought like lcc_rsi.pine, and indicator_table(bars) joining both. Every value at bar t may use data up to bar t only. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 5: Build the cross-check run (A3, K3)
In labs_fullcode/pipeline/indicators.py add run(symbol='AAPL', date='2025-12-31', tv_file=None) for `python -m pipeline indicators --symbol --date --tv`: compute indicator_table on the ADJUSTED research bars (from config data.start) and on the RAW reference bars from pipeline.validate.reference_bars (TradingView's default view); print a snapshot table (close, ema_fast, ema_slow, bb_upper, bb_middle, bb_lower, rsi) with columns Python ADJUSTED, Python RAW (TV default), TradingView (you) from Lab05-pine-indicators/tradingview_indicator_values.csv and 'diff vs RAW' in %; the trend, band and RSI reading; the number of bullish/bearish crosses and RSI alert events since 2024-06-01; PASS if the largest gap is <= 0.5%. Save _outputs/Lab05_<SYMBOL>_indicators.csv and chart Lab05_pine_preview.png (price, EMAs, bands, cross markers, RSI pane). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 6: Run the cross-check (A3, K3) (no prompt needed)
Run: python -m pipeline indicators --symbol AAPL --date 2025-12-31 (same as: python3 Lab05-pine-indicators/indicators_stage.py --symbol AAPL).
Step 7: Change the inputs and count the extra trading (K3, K7)
In a Python shell started in labs, use pipeline.indicators.lcc_trend_bands on pipeline.ingest.load_bars('AAPL', '2018-01-01', '2025-12-31') with fast/slow 20/50 and then 10/30, count bull_cross and bear_cross since 2024-06-01 for each, and estimate the extra trading cost of the faster setting at 3 bps per round trip on USD 100,000. Print a small table.

Open in Claude | Open in ChatGPT

Step 8: Create the alert the automation stage will use (A3, K7) (no prompt needed)
Restore EMA 20/50. Click Alert, choose Condition: LCC Trend > 'EMA bullish cross', trigger 'Once per bar close', and save. This alert type is what Lab 17 connects to the pipeline webhook.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 06 - Strategy Prototype: TradingView Strategy Tester, Paper Trading from the Chart and Pipeline Reconciliation

Folder Lab06-pine-strategy-tester/ - run: python3 Lab06-pine-strategy-tester/reconcile_stage.py

Step 1: Read the strategy code (A2, K3)

Explain labs_fullcode/Lab06-pine-strategy-tester/lcc_ma_crossover_strategy.pine for a finance professional: each strategy() setting, when an order is filled (process_orders_on_close = false means the NEXT bar's open), and how commission and slippage reduce the result. Do not change the file.

Open in Claude | Open in ChatGPT

Step 2: Run it in the Strategy Tester and record the results (A3, K7) (no prompt needed)

Open AMEX:SPY on 1D, paste the script into a new Pine Editor tab, save and Add to chart. Open the Strategy Tester panel and copy net profit, closed trades, percent profitable, profit factor, max drawdown, Sharpe, Sortino and commission paid into Lab06-pine-strategy-tester/tv_strategy_results.yaml (replace the nulls).

Step 3: Build the reconciliation stage (A2, A3, K3)

Create labs_fullcode/pipeline/reconcile.py. parse_pine(path) reads the //@version, the keyword settings inside strategy(...) and the input.int defaults (fastLen, slowLen) from Lab06-pine-strategy-tester/lcc_ma_crossover_strategy.pine. settings_checks(pine, cfg) returns PASS/FAIL rows: version 6; process_orders_on_close = false (next-bar-open fills = config strategy.execution next_open); commission_type cash_per_order with the value in config costs.tradingview; slippage > 0; default_qty_value 100 (% of equity); pyramiding 0. run(symbol='SPY', fast=None, slow=None) backtests the same rule with common.backtest.backtest(bars, cross_signal(ta_sma(fast), ta_sma(slow)), costs=pipeline.config.tradingview_cost_model(cfg), execution='next_open') and buy_and_hold, prints the Strategy Tester equivalents (net profit USD and %, closed and open trades, percent profitable, profit factor, max drawdown, Sharpe, Sortino, commission, buy and hold) next to the values in Lab06-pine-strategy-tester/tv_strategy_results.yaml with a difference column, the trade list, and four likely causes of gaps (adjusted vs unadjusted prices, 2018 history start, whole vs fractional shares, ticks vs bps slippage). Write _outputs/Lab06_reconciliation.yaml (plain Python numbers) and chart Lab06_crosscheck.png with common.plotting.plot_equity_and_drawdown. Exit code 1 if a settings check fails. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 4: Reconcile TradingView with the pipeline (A2, A3) (no prompt needed)

Run: python -m pipeline reconcile --symbol SPY --fast 50 --slow 200 (same as: python3 Lab06-pine-strategy-tester/reconcile_stage.py).

Step 5: Explain the differences (A3)

Here is my reconciliation output: <paste the table from python -m pipeline reconcile>. For each metric where TradingView and the pipeline differ by more than 5%, say which of these causes is most likely and how I could confirm it: adjusted vs unadjusted prices, the 2018 data start and SMA 200 warm-up, whole vs fractional shares, slippage in ticks vs bps.

Open in Claude | Open in ChatGPT

Step 6: Stress the costs and the speed of the rule (K7) (no prompt needed)

In TradingView change commission to 0.1% of order value, slippage to 5 ticks and the inputs to 20/50. Then run: python -m pipeline reconcile --fast 20 --slow 50.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 6: Stress the costs and the speed of the rule (K7) (no prompt needed)

Step 7: Put the strategy into action with TradingView Paper Trading (A3)

Act as my execution checklist for TradingView Paper Trading on AMEX:SPY with USD 100,000. Using the last pipeline close of 676.64, give the order-ticket fields and prices for (1) a market buy of 10 shares, (2) a limit buy 2% below the last price, (3) a bracket buy with take-profit +5% and stop-loss -3%; then say which status each order should show in the Orders and Positions tabs.

Open in Claude | Open in ChatGPT

Step 8: Manage the paper position from the chart (A3, K7) (no prompt needed)

Drag the stop-loss line to move it, close half the position, then close all. Read the History tab and the account balance. Note: a Pine strategy does not send orders to the Trading Panel by itself - automated orders go through alerts and webhooks (Lab 17) or the pipeline's execution stage (Lab 8).

Step 9: Optional - connect the Alpaca paper account in TradingView (A3) (no prompt needed)

In the Trading Panel choose Alpaca, log in, select the PAPER account and place one small market or limit order from the chart. Then run python -m pipeline check (online) to see the same position from the pipeline.

Step 10: Decide whether to promote the rule (A2) (no prompt needed)

Apply the strategy to SGX:D05 and compare. Decide whether SMA 50/200 should replace the pipeline's configured ema_cross 10/30 (do not change config.yaml yet - Labs 9 and 10 measure it properly).

Lab 07 - Signal Stage: a Library of Trend, Mean-Reversion, Momentum and Breakout Signals

Folder Lab07-technical-indicators/ - run: python3 Lab07-technical-indicators/signals_stage.py

Step 1: Agree the signal contract (A2) (no prompt needed)

Read the docstring of the reference Lab07-technical-indicators/solution/signals.py. Every signal returns a float Series of target positions (1 = long, 0 = flat) decided with information up to the CLOSE of bar t; the position is HELD from bar t+1. Warm-up bars (NaN indicators) are flat.

Step 2: Build the trend signals (A2, K3)

Create labs_fullcode/pipeline/signals.py. Helper _flat_until_ready(cond, *inputs) returns 1.0 where cond is true and 0.0 otherwise or while any input is NaN. Trend signals using pipeline.indicators: ema_cross(bars, fast=10, slow=30) long while ta_ema(fast) > ta_ema(slow); sma_cross(bars, fast=50, slow=200); trend_filter(bars, window=200) long while close > ta_sma(window). Each returns a float Series aligned to bars.index, using data up to bar t only. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Build mean-reversion, momentum and breakout signals (A2, K3)

In labs_fullcode/pipeline/signals.py add enter_exit(entry, exit_) - a stateful long/flat series that is 1 from an entry bar until the next exit bar (entry wins a tie, NaN treated as False); rsi_reversion(bars, n=2, entry=10, exit=70) = enter_exit(RSI < entry, RSI > exit); dip_buy(bars, n_down=3, hold=3) = long for `hold` bars after a bar completing >= n_down consecutive down closes; momentum(bars, lookback=126, threshold=0.0) long while the past lookback-bar return > threshold; donchian_breakout(bars, entry=55, exit=20) buy a close above the PRIOR entry-bar high (rolling max shifted 1), sell a close below the prior exit-bar low; bollinger_breakout(bars, window=20, k=2.0) buy a close above the upper band, sell below the middle band. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Add the registry, next-bar alignment and the look-ahead test (A2, K3)

In labs_fullcode/pipeline/signals.py add SIGNALS = {name: (function, family, default_params)} for the eight signals; target_position(name, bars, **params) (params override defaults, Series renamed to name); next_bar_position(target) = target.shift(1).fillna(0) (the position actually held); lookahead_test(name, bars, checks=25, seed=0, fn=None, **params): pick 25 random dates t in the last two-thirds of the sample and recompute the signal (a) on bars.loc[:t] only and (b) after multiplying every price AFTER t by 1.5 and by 0.5; count dates where any value up to t differs from the full-sample signal (any difference = look-ahead); describe_signal(target) with latest_target, entries, exposure, avg_hold_bars and last_change; latest_targets(symbols, name, params, start, end) returning a DataFrame of symbol, as_of, close, target, last_change. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 5: Build the signals command (A2)

In labs_fullcode/pipeline/signals.py add run(symbol='SPY') for `python -m pipeline signals --symbol`: for every signal in SIGNALS print family, params, latest target, entries, exposure, average holding bars, last change date and the look-ahead result (PASS or FAIL (n)); print one example of next-bar alignment for ema_cross (the close where the target changed and the first bar it is held); compute latest_targets for config universe.trade with config strategy name/params, print it and save _outputs/Lab07_targets.csv (input to the execution stage); save chart Lab07_signals.png (SPY 2024-2025 with green shading where ema_cross, rsi_reversion, momentum and donchian_breakout are long). Exit code 1 if any look-ahead test fails. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 6: Run the signal stage (A2, K3) (no prompt needed)

Run: python -m pipeline signals --symbol SPY (same as: python3 Lab07-technical-indicators/signals_stage.py).

Step 7: Catch a look-ahead bug on purpose (K3) (no prompt needed)

Temporarily change ema_cross to use bars['close'].shift(-1) (tomorrow's close) and rerun the stage. Then undo the change and rerun.

Step 8: Relate signal frequency to trading cost (K7)

Using the signal table from python -m pipeline signals, estimate for each signal the round trips per year over 2018-2025 and the annual cost drag at 3 bps per round trip on USD 100,000, assuming the full equity is traded. Rank the signals from most to least cost-sensitive and say which ones need a strong edge to survive costs.

Open in Claude | Open in ChatGPT

Lab 08 - Execution Stage: an Order Manager that Trades the Paper Account in Any Session

Folder Lab08-paper-orders/ - run: python3 Lab08-paper-orders/execution_stage.py

Step 1: Learn the order rules for each session (K5)

Using labs_fullcode/pipeline/clock.py, tell me which US session it is now in New York, what that is in Singapore time, and which order types, time_in_force values and extended_hours flag Alpaca will accept right now. Then explain why a market order is not allowed overnight. Do not change any file.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Build the planning logic: target vs actual (A3)

Create labs_fullcode/pipeline/execution.py with a dataclass OrderPlan(symbol, side, qty, current_qty, pending_qty, target_qty, price, reason) and a class OrderManager(broker, cfg, session=None). status() returns pipeline.clock.market_status (the simulator's replay clock if set; the real Alpaca clock online; session=... overrides it offline) and tells the simulator whether the market is open. positions() -> {symbol: qty}; pending() -> signed shares on open orders (+buy, -sell) via GetOrdersRequest(status=OPEN); quote(symbol, session) -> bid/ask/last: online StockLatestQuoteRequest (feed OVERNIGHT first in the overnight session, then IEX), falling back to the latest trade; offline the simulator's last price +/- 0.5 bp. plan(targets, session, units='weight'|'shares'): target_qty = floor(weight x equity / price) (or the share count), delta = target - held - pending, buy if delta > 0, sell (long only, never more than held) if delta < 0, otherwise print 'at target' and send nothing. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Build session-aware, idempotent order submission (A3, K5)

In labs_fullcode/pipeline/execution.py add client_id(plan, tag) = 'lcc-<tag>-<SYMBOL>-<side>-<qty>' (alphanumerics and dashes, max 48 characters; tag = decision date) so the same decision always gets the same id. request(plan, order_type, status, cid, quote, take_profit=0.05, stop_loss=0.03) builds an alpaca-py request: regular + market -> MarketOrderRequest DAY; regular + bracket (buys) -> MarketOrderRequest GTC with OrderClass.BRACKET, TakeProfitRequest and StopLossRequest; closed -> LimitOrderRequest GTC (queues for the next session); pre/after/overnight -> LimitOrderRequest DAY with extended_hours=True. Limit price = ask x (1 + live_paper.limit_buffer_bps) for buys, bid x (1 - buffer) for sells, rounded to 2 decimals. submit(plan, status, tag, order_type='auto'): first broker.get_order_by_client_id(cid) - if it exists print 'DUPLICATE ... not sent again' and return it; print the order (type, TIF, extended_hours, limit, quote and source, cid); send it with broker.submit_order. The broker comes from common.broker.get_broker (Alpaca TradingClient with paper=True, or PaperBrokerSim) - never create a live client. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 4: Add polling, protective-order handling and safe fallbacks (A3, K5)

In labs_fullcode/pipeline/execution.py add: poll(order, seconds) that re-reads the order every 2 seconds and prints the lifecycle (for example 'new -> filled') and the final order line, or '(still working - it stays on the book)'; cancel_protective(symbol) that cancels open SELL orders and bracket exit legs (GetOrdersRequest nested=True) before an exit, because they reserve the shares; in submit(), if an extended-hours order is rejected, print why (asset or account not enabled for overnight trading, price band, session change) and fall back to a GTC limit order with client id + '-q' that queues for the next session; rebalance(targets, tag, order_type, poll_seconds, units) = status -> plan -> submit -> poll; crypto_demo(broker, cfg) - ONLINE ONLY, clearly labelled DEMO: buy about USD 25 of BTC/USD with a GTC limit at ask + buffer, poll, then sell it back - only to show the fill loop working while US stocks are closed. run(...) and main(argv) for `python -m pipeline execute [--targets SPY=0.30 AAPL=0.10] [--order-type auto|market|limit|bracket] [--session ...] [--live-paper] [--crypto-demo] [--flatten] [--tag] [--reset]`: print [1] the clock and allowed orders, [2] the account, [3] plan -> orders, [4] an idempotency re-run of the same decision plus a deliberate duplicate submission, [5] positions and open orders (with bracket legs), the Alpaca dashboard address when online, and a RESULT line. With --live-paper or online, cap each weight at live_paper.max_position_pct. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Run the order manager in the current session (A3, K5) (no prompt needed)
Run: python -m pipeline execute --reset (same as: python3 Lab08-paper-orders/execution_stage.py --reset). Offline the session comes from the real clock, so the result depends on when you run it.
Step 6: Compare the sessions with the teaching override (K5) (no prompt needed)
Run: python -m pipeline execute --reset --session regular, then --session overnight, then --session closed.
Step 7: Bracket orders and a clean exit (A3, K5) (no prompt needed)
Run: python -m pipeline execute --reset --session regular --order-type bracket. Then flatten: python -m pipeline execute --flatten --session regular.
Step 8: Trade LIVE on the Alpaca paper account during class (A3, K5) (no prompt needed)
With paper keys in .env run: python -m pipeline execute --live-paper --targets SPY=0.05. Watch the clock line, the quote, the extended-hours limit order and its lifecycle for up to 30 seconds. Then open the Alpaca paper dashboard (app.alpaca.markets > Paper > Orders and Positions) and find the same client order id.
Step 9: Optional - watch a fill right now with the crypto demo (K5) (no prompt needed)
If US stocks are closed (weekend) or your extended-hours order is still working, run: python -m pipeline execute --live-paper --targets SPY=0.05 --crypto-demo. This is a DEMO of the fill loop only; the course and the pipeline remain stock-focused.
Step 10: Flatten the live paper account (A3) (no prompt needed)
Run: python -m pipeline execute --live-paper --flatten, then refresh the dashboard. Offline, run the same command without --live-paper.

Lab 09 - Evaluation Stage: Performance and Risk Metrics with a Benchmark Table

Folder Lab09-performance-metrics/ - run: python3 Lab09-performance-metrics/evaluate_stage.py

Step 1: Read the metric formulas (K3)
Explain each function in labs_fullcode/common/metrics.py with its formula and one sentence on what a portfolio manager learns from it; say which metrics need trade-level returns and which need a benchmark. Do not change the file.

Open in Claude | Open in ChatGPT

Step 2: Build the evaluation stage (A3, K3)
Create labs_fullcode/pipeline/evaluate.py. evaluate_strategy(bars, name, costs, start, bench_ret, **params) backtests pipeline.signals.target_position(name, bars, **params) with common.backtest.backtest (next-bar open fills, costs = pipeline.config.cost_model(cfg)), slices returns and position from `start` and returns common.metrics.summary(...) with trade net returns and the benchmark. run(symbol='SPY'): start = first date the 200-day SMA exists (so every signal is warm), benchmark = buy_and_hold sliced from start; evaluate every signal in SIGNALS; print a table (rows = strategies + buy & hold, columns CAGR, Ann. volatility, Sharpe, Sortino, Max drawdown, Calmar, Trades, Win rate, Profit factor, Exposure, Turnover, Alpha, Beta) ranked by Sharpe and formatted with common.metrics.format_table; list which strategies beat buy and hold on Sharpe and how many have a smaller drawdown; print 'Sharpe by hand' for trend_filter (mean / stdev x sqrt(252)), the worst day and SPY's drawdown trough; save _outputs/Lab09_metrics.csv and chart Lab09_metrics.png (top 4 by Sharpe plus buy & hold, equity and drawdown). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Run the evaluation (A3) (no prompt needed)
Run: python -m pipeline evaluate --symbol SPY (same as: python3 Lab09-performance-metrics/evaluate_stage.py).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Run the evaluation (A3) (no prompt needed)
Step 4: Check one metric by hand (K3) (no prompt needed)
Read the 'Sharpe by hand' line and confirm it equals the table.
Step 5: Interpret trade and market metrics (A3, K3)
Using labs_fullcode/_outputs/Lab09_metrics.csv, write a 120-word note for Lion City Capital's investment committee comparing ema_cross, trend_filter and SPY buy & hold on return, risk and risk-adjusted return. Quote the numbers, explain win rate versus profit factor, and do not recommend live trading.

Open in Claude | **Open in ChatGPT**

Step 6: Locate the worst periods (K3) (no prompt needed)
Read the worst-day and drawdown-trough line and find those dates on _outputs/Lab09_metrics.png.
Step 7: Add a metric to the stage (K3)
In labs_fullcode/pipeline/evaluate.py add an 'Ulcer index' column = sqrt(mean(drawdown% squared)) computed from common.metrics.drawdown_series of each strategy's returns, without changing common/metrics.py. Show it in the printed table and the CSV, and explain in one sentence what it adds to max drawdown. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Lab 10 - Strategy-Search Stage: Grid Search, Out-of-Sample Ranking and Decay

Folder Lab10-strategy-grid-search/ - run: python3 Lab10-strategy-grid-search/search_stage.py

Step 1: Define the search space (A2, K2) (no prompt needed)
Read GRIDS in Lab10-strategy-grid-search/solution/search.py: sma_cross fast x slow (24 valid pairs), rsi_reversion n x entry x exit (36), bollinger_breakout window x k (16) - 76 sets.
Step 2: Build the grid search (A2, A4)
Create labs_fullcode/pipeline/search.py. IS_END = '2021-12-31', OOS_START = '2022-01-01'. GRIDS = {'sma_cross': {'fast': [5,10,20,30,50], 'slow': [50,100,150,200,250]}, 'rsi_reversion': {'n': [2,3,5,14], 'entry': [10,20,30], 'exit': [50,60,70]}, 'bollinger_breakout': {'window': [10,20,30,50], 'k': [1.0,1.5,2.0,2.5]}}. grid_points(name) yields parameter dicts (skip sma_cross with fast >= slow). search(bars, costs) runs ONE backtest per set with pipeline.signals.target_position and common.backtest.backtest (signals are causal, so one run covers both windows), then records IS and OOS Sharpe and CAGR (common.metrics) and trades; add IS_rank and OOS_rank (method='min', NaN last) and decay = OOS - IS Sharpe. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Add ranking, robustness and the decay report (A4, K2)
In labs_fullcode/pipeline/search.py add robust_choice(ma) - for the sma_cross grid, the cell with the best AVERAGE in-sample Sharpe over itself and its neighbours - and run(symbol='SPY') for `python -m pipeline search`: print buy-and-hold Sharpe IS and OOS; a decay report with the best IS set per family (IS -> OOS Sharpe, decay, OOS rank of 76, trades); the top 5 by IS Sharpe; the Spearman rank correlation between IS and OOS Sharpe; the average decay of the top 5; the share of all sets that beat buy and hold OOS; the single best MA cell versus the robust MA region; and a suggested config.yaml strategy line (do NOT edit the config automatically). Save _outputs/Lab10_grid_results.csv, _outputs/Lab10_decay_report.csv and chart Lab10_grid_heatmap.png (IS and OOS Sharpe heatmaps of the MA grid with values in each cell). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 4: Run the search (A4) (no prompt needed)
Run: python -m pipeline search --symbol SPY (same as: python3 Lab10-strategy-grid-search/search_stage.py).
Step 5: Read the overfitting evidence (A4, K2) (no prompt needed)
Read the top-5 table and the three summary lines.
Step 6: Prefer a robust region to a single peak (A2, A4) (no prompt needed)
Compare the single best MA cell with the robust MA region and open _outputs/Lab10_grid_heatmap.png.
Step 7: Decide whether to promote anything (A2, K2)
Using labs_fullcode/_outputs/Lab10_decay_report.csv and labs_fullcode/_outputs/Lab10_grid_results.csv, argue for and against replacing the pipeline's configured ema_cross 10/30 with the best sma_cross or bollinger_breakout set. Mention the number of sets tried (76), the decay, the rank correlation and what the diagnostics stage (Lab 18) should check before any change. Finish with a one-line recommendation.

Open in Claude | Open in ChatGPT

Lab 11 - ML-Signal Stage: a Walk-Forward Classifier from Probability to Position

Folder Lab11-ml-signal-classifier/ - run: python3 Lab11-ml-signal-classifier/ml_stage.py

Step 1: Build causal features and the label (K4)
Create labs_fullcode/pipeline/ml.py with build_features(bars): features known at the close of day t only - ret_lag1/2/3/5 (r.shift(lag-1), so ret_lag1 = today's return), mom_5, mom_21, rsi_14 and rsi_2 (pipeline.indicators.ta_rsi), macd_hist / close (ta_macd), Bollinger %B from ta_bb, vol_21 annualised, dist_sma50 and dist_sma200, volume z-score over 21 days; label target = 1 if close t+1 > close t. Drop the last row (no label yet) and rows with NaN. The label may use shift(-1); no FEATURE may. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Add walk-forward validation (K4, K6)
In labs_fullcode/pipeline/ml.py add MODELS = {'Logistic regression': StandardScaler + LogisticRegression(C=0.1, max_iter=1000), 'Random forest': RandomForestClassifier(n_estimators=300, max_depth=4, min_samples_leaf=20, random_state=42), 'XGBoost': XGBClassifier(n_estimators=200, max_depth=3, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, random_state=42, eval_metric='logloss') imported lazily}, and walk_forward_proba(X, y, make, splits=5) using TimeSeriesSplit(n_splits=5, gap=1): fit on each training block, predict_proba on the following test block, return the out-of-sample probabilities and the last model. Never fit on future data. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Turn probability into a position and backtest it (A4)
In labs_fullcode/pipeline/ml.py add position_from_probability(prob, threshold, index=None) = 1.0 where P(up) > threshold (decided at close t, so the backtester fills it at the next open), and run(symbol='SPY', threshold=0.55, models=None) for `python -m pipeline ml --threshold --models`: print samples, features, base rate and the five folds; out-of-sample accuracy, precision, AUC and share predicted up per model, with the naive 'always up' benchmark; backtest each model's positions from the first test date with common.backtest.backtest and the config cost model against buy and hold (CAGR, Sharpe, max drawdown, exposure, trades, turnover); print a VERDICT ('NO EDGE' if the best AUC < 0.53); save _outputs/Lab11_ml_probabilities.csv and chart Lab11_ml_strategy.png (equity curves and XGBoost feature importance from the last fold). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 4: Run the ML stage (A4, K4) (no prompt needed)
Run: python -m pipeline ml --threshold 0.55 (same as: python3 Lab11-ml-signal-classifier/ml_stage.py).
Step 5: Read the strategy result (A4) (no prompt needed)
Read the backtest table and the VERDICT.
Step 6: Test the threshold - and notice the snooping risk (A4, K6) (no prompt needed)
Run: python -m pipeline ml --threshold 0.5 --models 'Logistic regression', then --threshold 0.6.
Step 7: Prove that leakage is detectable (K6)
In a Python shell started in labs, import pipeline.ml, wrap build_features so it also adds a feature 'leak' = tomorrow's return (bars['close'].pct_change().shift(-1)), run walk_forward_proba with the 'Logistic regression' model and print the out-of-sample AUC. Explain why the result proves the feature leaks the future, then discard the code.

Open in Claude | Open in ChatGPT

Step 8: Inspect the feature importance (K4) (no prompt needed)
Open _outputs/Lab11_ml_strategy.png. Which features does XGBoost rely on, and does that make economic sense?

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 12 - Regime-Filter Stage: PCA and K-Means as a Gate on the Signals

Folder Lab12-market-regimes/ - run: python3 Lab12-market-regimes/regime_stage.py

Step 1: Build PCA and the regime features (K4)
Create labs_fullcode/pipeline/regime.py. regime_features(closes) builds backward-looking SPY features: vol_21 (annualised 21-day volatility), ret_21, ret_63, dist_sma200 (close / ta_sma(200) - 1) and corr_spy_tlt (63-day rolling correlation of SPY and TLT returns), dropping NaN rows. fit_regimes(train, k) fits StandardScaler + KMeans(n_clusters=k, n_init=20, random_state=42) and names clusters by average vol_21: 'Low vol', 'Moderate vol', 'High vol (stress)' for k=3, otherwise 'Regime i (vol rank)' with the highest named 'High vol (stress)'. In run() also fit sklearn PCA on standardised daily returns of config universe.trade (pipeline.ingest.load_closes) and print variance explained by PC1-PC4 and the loadings. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 2: Add walk-forward labels and the gate (K6, A4)
In labs_fullcode/pipeline/regime.py add walk_forward_regimes(feat, k, first_year=2020): for each calendar year fit_regimes on data up to 31 December of the previous year only and label that year's days; regime_gate(target, regimes, blocked=('High vol (stress)',)) forces the target to 0 on blocked days (the regime is known at the close, so the gate acts on the next bar like any signal). run(k=3) for `python -m pipeline regime --clusters`: print [1] PCA, [2] full-sample cluster profiles (labelled hindsight, description only) with next-day SPY Sharpe and trend-filter Sharpe by regime, [3] walk-forward regime counts and the latest regime and gate state, [4] gated versus ungated CAGR, Sharpe, max drawdown, exposure and trades from 2020 for the configured strategy, trend_filter and dip_buy (backtest with config costs). Save _outputs/Lab12_regime_labels.csv and chart Lab12_regimes.png (PCA variance, SPY coloured by walk-forward regime, gated vs ungated equity). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Run the regime stage and read the PCA (A4, K4) (no prompt needed)
Run: python -m pipeline regime --clusters 3 (same as: python3 Lab12-market-regimes/regime_stage.py).
Step 4: Read the regime profiles (K4, K6) (no prompt needed)
Read section [2]. In which regime does the trend filter work, and where does it fail?
Step 5: Check the walk-forward labels (K6) (no prompt needed)
Read section [3]. Why does the walk-forward model label far fewer stress days than the full-sample fit?
Step 6: Evaluate the gate after costs (A4)
Using the gated versus ungated table from python -m pipeline regime, write a short recommendation to the risk team on whether to switch the regime gate on for ema_cross, trend_filter and dip_buy. Quote Sharpe and max drawdown, mention that the gate is itself a model fitted on few stress days, and say which test from Lab 13 should decide.

Open in Claude | Open in ChatGPT

Step 7: Try four clusters (A4) (no prompt needed)
Run: python -m pipeline regime --clusters 4 and compare the profiles.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 13 - A/B-Test Stage: a Statistical Decision before Capital Moves

Folder Lab13-ab-testing/ - run: python3 Lab13-ab-testing/abtest_stage.py

Step 1: State the hypotheses (A5, K2) (no prompt needed)
Write H0: both variants have the same expected daily return and Sharpe; H1: they differ. Choose alpha = 0.05 and note how many variants the desk has tried this quarter (default 20).
Step 2: Build the statistics (A5, K2)
Create labs_fullcode/pipeline/abtest.py. block_bootstrap_sharpe_diff(a, b, n_boot=2000, block=21, seed=7) resamples blocks of 21 consecutive days with numpy default_rng(seed) and returns Sharpe(A) - Sharpe(B) for each draw (annualised with sqrt(252)). ab_test(ra, rb, alpha=0.05, tried=20) returns the Welch t-test (scipy.stats.ttest_ind equal_var=False), the paired t-test (ttest_rel) and mean daily difference, the Sharpe difference and its (1 - alpha) bootstrap CI, the share of draws where A beats B, reject_h0 = paired p < alpha AND the CI excludes 0, and the Bonferroni threshold alpha / tried. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 3: Build the A/B command (A5)
In labs_fullcode/pipeline/abtest.py add run(a='trend_filter', b='dip_buy', alpha=0.05, tried=20, symbol='SPY') for `python -m pipeline abtest --a --b --alpha --tried --symbol` (a and b chosen from pipeline.signals.SIGNALS): backtest both with the config cost model, use the SAME window from the first valid 200-day SMA, print CAGR, volatility, Sharpe, max drawdown, exposure and trades, the three tests, a DECISION line ('REJECT H0 - ...' or 'DO NOT REJECT H0 - <winner> looks better on paper, but the evidence is not reliable; keep both on paper') and the Bonferroni caution; write _outputs/Lab13_abtest_decision.yaml and chart Lab13_ab_test.png (both equity curves and the bootstrap histogram with the CI). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 4: Run the A/B test (A5) (no prompt needed)
Run: python -m pipeline abtest --a trend_filter --b dip_buy --alpha 0.05 (same as: python3 Lab13-ab-testing/abtest_stage.py).
Step 5: Compare the independent and paired tests (K2) (no prompt needed)
Read tests 1 and 2. Why does the paired design suit two strategies measured on the same days?
Step 6: Read the bootstrap CI and the decision (A5)
Rewrite this A/B test result for two portfolio managers who are not statisticians, in under 100 words: <paste the three test lines and the DECISION line>. Explain why 'A looks better' is not the same as 'A is better', and what evidence would change the decision.

Open in Claude | **Open in ChatGPT**

Step 7: Test the pipeline's own strategy (A5, K2) (no prompt needed)
Run: python -m pipeline abtest --a ema_cross --b trend_filter.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 14 - Pairs Stage: a Hedged, Market-Neutral Statistical-Arbitrage Sleeve

Folder Lab14-pairs-trading/ - run: python3 Lab14-pairs-trading/pairs_stage.py

Step 1: Read the method (K6) (no prompt needed)
Read the docstring of Lab14-pairs-trading/solution/pairs.py: the formation period 2018-2022 is used only to estimate the hedge ratio and test cointegration; the trading period 2023-2025 is out of sample with the hedge ratio held fixed.
Step 2: Build the cointegration tools (K6)
Create labs_fullcode/pipeline/pairs.py with numpy-only statistics: ols(y, X) returning coefficients, standard errors and residuals; adf_t(series, lags=1) - the ADF t-statistic of g in d e_t = g e_(t-1) + phi d e_(t-1); engle_granger(log_a, log_b) - step 1 OLS log A = a + beta log B, step 2 adf_t on the residual - returning (alpha, beta, adf_t); half_life(spread) from an AR(1) fit of spread changes; EG_CRIT = {'1%': -3.90, '5%': -3.34, '10%': -3.04} (MacKinnon, two variables with constant). FORMATION = ('2018-01-01', '2022-12-31'), TRADING = ('2023-01-01', '2025-12-31'). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 3: Build the spread trading rules and leg targets (A4)
In labs_fullcode/pipeline/pairs.py add trade_pair(closes, a, b, alpha, beta, entry, exit_, window, cost_bps): spread = log A - beta log B - alpha; z = (spread - rolling mean) / rolling std over window; state machine: short spread (-1: short A, long B) when z > +entry, long spread (+1) when z < -entry, exit when |z| < exit_; held = state.shift(1) (decided at close t, held over t+1); gross = held x w x (rA - beta rB) with w = 1 / (1 + |beta|); costs = |change in held| x w x (1 + |beta|) x cost_bps per leg per side; return spread, z, held, state, gross, net. leg_targets(state, a, b, beta) -> {a: state x w, b: -state x beta x w}. run(pair=('KO', 'PEP'), entry=2.0, exit_=0.5, window=60, cost_bps=5.0) for `python -m pipeline pairs --pair --entry --exit --window --cost-bps`: screen config universe.pairs (correlation of returns, beta, ADF t, cointegrated at 5%?), print the chosen pair's hedge ratio and half-life (warn if not cointegrated), trade 2023-2025 using 120 days of history for the z warm-up, print gross, net and SPY return, CAGR, volatility, Sharpe and max drawdown, trades, time in market, beta and correlation to SPY, the latest leg targets, and a note that short legs need a margin account while the automation stages are long only. Save chart Lab14_pairs_trading.png (normalised prices, z with bands and open positions, equity vs SPY). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 4: Run the pair screen (A4, K6) (no prompt needed)
Run: python -m pipeline pairs (same as: python3 Lab14-pairs-trading/pairs_stage.py). Compare correlation with the ADF t-statistic for each pair.
Step 5: Interpret the hedge ratio (K6) (no prompt needed)
Read section [2]. Explain in one sentence why the hedge ratio, not a 1:1 position, makes the spread stationary.
Step 6: Evaluate the out-of-sample result (A4) (no prompt needed)
Read section [3] and [4]: trades, time in market, beta to SPY and the latest leg targets.
Step 7: Test a failed pair and other thresholds (A4, K6) (no prompt needed)
Run: python -m pipeline pairs --pair JPM BAC --entry 1.5 --exit 0. Explain why trading a pair that failed the cointegration test is a statistical error and why trying many pairs and thresholds is data snooping.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Run the pair screen (A4, K6) (no prompt needed)
Step 8: Study the chart (A4)
Looking at labs_fullcode/_outputs/Lab14_pairs_trading.png and the stage output, explain to the head of trading how the desk should re-test cointegration on a rolling basis and size each pair small before this sleeve could ever be traded, and why it stays a paper sleeve while the automation stages are long only.

Open in Claude | Open in ChatGPT

Lab 15 - Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even

Folder Lab15-trading-costs/ - run: python3 Lab15-trading-costs/backtest_stage.py

Step 1: Read the cost model (K7)
Explain CostModel.trade_cost in labs_fullcode/common/backtest.py with a worked example: a USD 95,000 buy and the matching sell of SPY at 676.64 under the alpaca_retail preset. Show each component in dollars. Do not change the file.

Open in Claude | Open in ChatGPT

Step 2: Build the engine helpers (A7)
Create labs_fullcode/pipeline/backtest.py. run_backtest(bars, name, costs=None, execution='next_open', **params) = common.backtest.backtest(bars, pipeline.signals.target_position(name, bars, **params), costs, execution) - the single entry point later stages use. break_even_bps(bars, signal, lo=0, hi=1000): bisection (30 steps) on a one-way cost expressed as CostModel(slippage_bps=x) until net CAGR = 0 (return 0 if gross CAGR <= 0). parse_custom(['spread_bps=3', 'slippage_bps=3']) -> CostModel('Custom route', ...). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Build the scenario report (A7, A8)
In labs_fullcode/pipeline/backtest.py add run(strategies=None, symbol='SPY', custom=None) for `python -m pipeline backtest --strategies --symbol --custom`: default strategies dip_buy and trend_filter; scenarios from config costs.scenarios plus the custom one; for each strategy and scenario print gross CAGR, net CAGR, cost drag, net Sharpe, executions and dollars of commission, spread, slippage, reg fees and total (from BacktestResult.cost_summary()); print each strategy's break-even bps per side next to its number of round trips and average gross trade in bps; print next-bar OPEN versus next-bar CLOSE CAGR and Sharpe with alpaca_retail costs; save _outputs/Lab15_cost_scenarios.csv and chart Lab15_costs.png (equity by scenario for the first strategy with the gross curve dashed, and a stacked bar of where the costs go). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 4: Run the cost scenarios (A7, A8) (no prompt needed)
Run: python -m pipeline backtest --strategies dip_buy trend_filter (same as: python3 Lab15-trading-costs/backtest_stage.py).
Step 5: Analyse the cost breakdown (K7) (no prompt needed)
For the Alpaca scenario, which component is largest even though commission is zero?
Step 6: Interpret break-even and execution timing (A8, K7) (no prompt needed)
Compare each break-even cost with the average trade, then read the execution-timing lines.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Run the cost scenarios (A7, A8) (no prompt needed)
Step 7: Configure your own execution route (A7) (no prompt needed)
Run: python -m pipeline backtest --custom spread_bps=3 slippage_bps=3 and record the new net CAGR for both strategies.

Lab 16 - Automation Stage: a Scheduled, Risk-Checked Forward Test and Live Paper Trading

Folder Lab16-paper-trading-bot/ - run: python3 Lab16-paper-trading-bot/run_stage.py

Step 1: Build the risk layer and the kill switch (A7)
Create labs_fullcode/pipeline/risk.py with class RiskManager(cfg). kill_switch_on() = the file named by risk.kill_switch_file exists under labs. check(broker, status, symbol, wants_buy, allow_queue=False) returns (ok_to_trade, ok_to_buy, notes): kill switch -> (False, False, ['KILL_SWITCH present - trading halted']); session 'closed' -> blocked with 'market closed' unless allow_queue (then note 'order will queue (GTC) for the next session'); equity / last_equity - 1 <= -risk.max_daily_loss_pct -> no new buys; symbol not in risk.allowed_symbols -> no buy; account trading_blocked -> no buy; otherwise ['all checks passed']. cap_weight(weight, equity, price) applies max_position_pct and max_order_value. kill_main(argv) for `python -m pipeline kill --on | --off | --status`: --on writes the file with a timestamp; --off MOVES it to _state/kill_switch_history/KILL_SWITCH.<timestamp> (an audit record - never delete it). Risk is deterministic code; no model may override it. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 2: Build one decision cycle (A7)
Create labs_fullcode/pipeline/runner.py. decide(bars, cfg) = the configured strategy's target from pipeline.signals.target_position on COMPLETED bars only, returning bar_date, close, signal and (for ema_cross) ema_fast and ema_slow. run_cycle(broker, om, risk, bars, run_time, cfg, weight, allow_queue=False, poll_seconds=0): read position and pending shares from the pipeline.execution.OrderManager; wants_buy = signal 1 and nothing held or pending; wants_sell = signal 0 and shares held; run RiskManager.check; if allowed call om.rebalance({symbol: weight}) or om.rebalance({symbol: 0.0}) with tag = the bar date (one decision per bar, so a double-fired job cannot double-trade); otherwise action BLOCKED if a trade was wanted; return a log row with run_time, bar_date, symbol, close, ema_fast, ema_slow, signal, position_before, action, qty, order_id, status, fill_price, equity, checks. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Build the forward-test replay and the logs (A8)
In labs_fullcode/pipeline/runner.py add replay(cfg, start, end): offline forward test on PaperBrokerSim (state_file bot_sim.json, reset) - for each 2025 session set the simulator clock to 09:35, mark the price at today's open, run_cycle on bars up to YESTERDAY's close with weight risk.max_position_pct, then process the day's bar and roll end_of_day; stop if the kill switch appears. Write _outputs/trade_log.csv (fresh, one row per session) and _outputs/Lab16_equity_log.csv (date, equity, position, close), print every non-HOLD row, the number of orders, start and end equity and SPY buy and hold over the same days, and save chart Lab16_forward_test.png. once(cfg): one cycle; offline it acts at the open after the last bundled bar and appends to trade_log.csv; online it appends to _outputs/live_paper_log.csv. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Add the live paper loop and scheduling (A7)
In labs_fullcode/pipeline/runner.py add live_loop(cfg, interval, max_cycles): on the Alpaca PAPER account (or the simulator with a clear message if no keys) run up to max_cycles cycles: print the clock line, stop with 'KILL SWITCH ON - no orders sent; loop stopped' if the kill switch exists, run_cycle with weight live_paper.max_position_pct, allow_queue=True and poll_seconds live_paper.poll_seconds online (the OrderManager sends extended-hours limit orders outside regular hours), append to _outputs/live_paper_log.csv, print the row and every position with unrealised P&L, then sleep interval seconds; finish with the dashboard address. main(argv): --loop --live-paper -> live_loop; --loop offline -> replay; --loop online -> live_loop; otherwise once. schedule_main(argv) for `python -m pipeline schedule`: print and write to _outputs/Lab16_schedule/ a crontab line with CRON_TZ=America/New_York for schedule.run_at_exchange_time Mon-Fri (with the Singapore time in US summer and winter), a launchd plist (com.lioncitycapital.pipeline, WorkingDirectory labs) and a Windows schtasks command, each running `python -m pipeline run --once` from labs and appending to _outputs/scheduler.log. Do not install anything. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 5: Run the 2025 forward test (A8, K7) (no prompt needed)
Run: python -m pipeline run --loop (same as: python3 Lab16-paper-trading-bot/run_stage.py --loop). Offline this replays 2025 day by day; each order fills at that day's open in the simulator.
Step 6: Audit the decision log (A8)
Read labs_fullcode/_outputs/trade_log.csv and write an audit summary for the compliance officer: number of sessions, decisions by action, every order with its bar date, fill price and equity after the fill, and any row where a risk check did not say 'all checks passed'. Do not change any file.

Open in Claude | **Open in ChatGPT**

Step 7: Test the kill switch (A7) (no prompt needed)
Run: python -m pipeline kill --on, then python -m pipeline run --once, then python -m pipeline kill --off.
Step 8: Run the strategy LIVE on the paper account (A7, A8, K7) (no prompt needed)
With paper keys in .env, during class run: python -m pipeline run --loop --interval 60 --live-paper --max-cycles 5. Watch decision -> order -> fill -> position -> log for each cycle. After two or three cycles, in a SECOND terminal run python -m pipeline kill --on and watch the next cycle halt. Then run python -m pipeline kill --off and open _outputs/live_paper_log.csv and the Alpaca dashboard.
Step 9: Schedule the runner (A7) (no prompt needed)
Run: python -m pipeline schedule and read the three entries. Only install one if your instructor approves (crontab -e, launchctl load, or schtasks in a Command Prompt).

Lab 17 - Event-Driven Automation: TradingView Webhook to Signal, Risk and Execution

Folder Lab17-tradingview-webhook/ - run: python3 Lab17-tradingview-webhook/webhook_stage.py

Step 1: Set the shared secret (A7) (no prompt needed)
In labs_fullcode/.env set WEBHOOK_SECRET to a long random string (for example the output of python3 -c "import secrets; print(secrets.token_urlsafe(24))"). Without it the receiver uses a demo secret and prints a warning.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1: Set the shared secret (A7) (no prompt needed)

Step 2: Build alert validation (A7, K5)

Create labs_fullcode/pipeline/webhook.py with class Gateway(cfg, session=None): secret from pipeline.config.secret(cfg, 'webhook_secret') (demo value 'lcc-demo-secret' if unset), broker = common.broker.get_broker(state_file='webhook_sim.json', reset=True), an OrderManager, a RiskManager and a set of processed alert ids. handle(payload) -> (http_code, body): 401 if hmac.compare_digest of the secret fails; ticker 'NASDAQ:AAPL' -> 'AAPL'; 422 if the ticker is not in risk.allowed_symbols, the action is not buy/sell, qty is not a number or not 0 < qty <= risk.max_order_qty; alert id = payload 'id' or a hash of ticker, action, qty and time; a repeated id returns 200 'duplicate ignored' (TradingView retries are safe). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Route valid alerts through signal, risk and execution (A7, K7)

In Gateway.handle add: pipeline_signal(ticker) = the configured strategy's latest target from pipeline.signals for that ticker; if webhook.require_signal_agreement is true and the alert disagrees, return 409; a sell with no position returns 422 (long only); RiskManager.check (kill switch, session, daily loss) failing returns 423 'blocked by risk'; otherwise target shares = held + qty for a buy or held - qty for a sell and call OrderManager.rebalance({ticker: target}, tag='tv<alert id>', units='shares') so the order is session-aware and idempotent; return 200 with order id, status, side, qty, fill price, pipeline_signal and signal_agrees. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 4: Build the server, the simulator and the self-test (A7)

In labs_fullcode/pipeline/webhook.py add a standard-library http.server receiver (no Flask): POST webhook.path parses JSON (400 'body is not valid JSON' otherwise), calls Gateway.handle, appends a row to _outputs/Lab17_webhook_log.csv (time, client, alert_id, ticker, action, qty, http, result, pipeline_signal, order_id, status, detail; header written fresh when the server starts) and replies JSON; GET /health returns status, broker mode and kill switch. serve(cfg, host, port, max_requests) handles one request at a time; offline it uses a simulated regular session. test_alerts(secret) returns seven cases with expected codes: valid BUY 5 SPY (200), valid SELL 5 AMEX:SPY (200), wrong secret (401), TSLA (422), qty 5000 (422), malformed text (400), a retry of alert 1 (200 duplicate). send_test_alerts(url, secret) posts them with urllib and prints OK or MISMATCH; selftest(cfg) runs the server on a free port in a thread, sends the alerts and prints 'RESULT: n/7'. main(argv) for `python -m pipeline webhook [--port] [--max-requests] [--selftest]` and alert_sim_main(argv) for `python -m pipeline alert-sim --url --secret`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 5: Run the self-test (A7, K5) (no prompt needed)

Run: python -m pipeline webhook --selftest.

Step 6: Run the receiver and the simulator in two terminals (A7) (no prompt needed)

Terminal 1: python -m pipeline webhook --port 8080 (same as: python3 Lab17-tradingview-webhook/webhook_stage.py --port 8080). Open http://127.0.0.1:8080/health in a browser. Terminal 2: python3 Lab17-tradingview-webhook/send_test_alert.py --url http://127.0.0.1:8080/webhook. Stop the receiver with Ctrl+C.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Run the self-test (A7, K5) (no prompt needed)
Step 7: Let the pipeline's signal veto an alert (A7, K7) (no prompt needed)
In pipeline/config.yaml set webhook.require_signal_agreement: true and rerun python -m pipeline webhook --selftest. Set it back to false.
Step 8: Prove the kill switch reaches the webhook (A7) (no prompt needed)
Run: python -m pipeline kill --on, then python -m pipeline webhook --selftest, then python -m pipeline kill --off.
Step 9: Optional - connect a real TradingView alert (paid plan) (A7, K5)
Review labs_fullcode/Lab17-tradingview-webhook/tradingview_alert_message.json and lcc_webhook_strategy.pine. Explain which TradingView placeholders fill ticker, action, qty and id, why the id makes retries safe, and three ways to keep the shared secret from leaking (who can see alert messages, chart sharing, logs). Do not change the files.

Open in Claude | **Open in ChatGPT**

Lab 18 - Diagnostics Stage: Automated Bias Checks before Go-Live

Folder Lab18-diagnose-bias/ - run: python3 Lab18-diagnose-bias/diagnose_stage.py

Step 1: Run the suspicious backtest (A9) (no prompt needed)
Run: python3 Lab18-diagnose-bias/buggy_backtest.py. Record the CAGR, Sharpe ratio and win rate.
Step 2: Find the problems (A9, K8)
Review labs_fullcode/Lab18-diagnose-bias/buggy_backtest.py as a model-risk reviewer. For each line marked CHECK 1 to 3, explain the bias, how it inflates the result and the correct fix. Do not run or change the file.

Open in Claude | **Open in ChatGPT**

Step 3: Build the look-ahead checks (A9, K8)
Create labs_fullcode/pipeline/diagnose.py. buggy_signal(bars) reproduces the exhibit (5-day rolling mean with center=True, long when it rises); trailing_signal(bars) is the fix (trailing window). In run(strategy=None, trials=None) add check 1, look-ahead in FEATURES, using pipeline.signals.lookahead_test (25 truncation truncation and future-shock checks) on the configured strategy and on the exhibit (fn=buggy_signal); and check 2, look-ahead in EXECUTION: the strategy's Sharpe with same-bar returns (target_t x return_t) versus the pipeline's next-bar backtest, plus the exhibit fixed one problem at a time: 0 as submitted (centred, same bar), 1 trailing window same bar, 2 + next-bar execution (common.backtest.backtest), 3 + alpaca_retail costs, and SPY buy and hold, as a CAGR / Sharpe / max drawdown table. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Build the cost, overfitting and survivorship checks (A9, K8)
In labs_fullcode/pipeline/diagnose.py add: check 3 cost drag - gross vs net CAGR from cost_summary(), break-even from pipeline.backtest.break_even_bps versus the assumed one-way cost (half spread + slippage of the config preset): PASS if break-even >= 3 x assumed and drag < 25% of gross; check 4 overfitting - random_strategies(spy_returns, 500, seed=1) (skill-free long/flat signals switching with 10% probability a day, traded next bar), the best in-sample Sharpe among the first N where N = the number of sets in _outputs/Lab10_grid_results.csv (default 76), compared with the strategy's 2018-2021 Sharpe, and its 2022-2025 decay; check 5 survivorship / hindsight - the universe was chosen from today's listed symbols, and an equal-weight NVDA/MSFT/AAPL 'AI basket' versus SPY. Print each check as [PASS|WARN|FAIL|INFO] check subject evidence, a VERDICT ('BLOCK - fix the FAIL items' if the strategy has a FAIL, else 'CLEAR for paper trading - review every WARN'), write _outputs/Lab18_diagnostics.csv and charts Lab18_bias_comparison.png and Lab18_overfitting.png. main(argv): --strategy, --trials, --strict (exit 1 on BLOCK). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 5: Run the diagnostics (A9) (no prompt needed)
Run: python -m pipeline diagnose (same as: python3 Lab18-diagnose-bias/diagnose_stage.py).
Step 6: Read the exhibit fixed one problem at a time (A9, K8) (no prompt needed)
Read the fix table and open _outputs/Lab18_bias_comparison.png.
Step 7: Read the overfitting and survivorship evidence (K8) (no prompt needed)
Read the INFO row and open _outputs/Lab18_overfitting.png. Then run python -m pipeline diagnose --strategy dip_buy.
Step 8: Use the stage as a gate and write the committee note (A9)
Using labs_fullcode/_outputs/Lab18_diagnostics.csv, write a 4-sentence note for the investment committee: why the junior quant's Sharpe of 4.28 is not credible, which automated check caught each problem, what the configured ema_cross strategy's WARN items mean, and what must happen before any live capital.

Open in Claude | Open in ChatGPT

Lab 19 - Monitoring and Reporting Stage: Automated Tearsheet and Backtest-vs-Forward Report

Folder Lab19-performance-report/ - run: python3 Lab19-performance-report/report_stage.py

Step 1: Make sure the forward-test logs exist (A8) (no prompt needed)
The stage reads _outputs/Lab16_equity_log.csv and _outputs/trade_log.csv from the Lab 16 runner; if either is missing it runs the 2025 replay itself. Run python -m pipeline run --loop if you want fresh logs.
Step 2: Build the backtest-vs-forward comparison (A8)
Create labs_fullcode/pipeline/report.py. load_forward_test(cfg) reads the equity log and the BUY/SELL rows of trade_log.csv whose run_time falls in 2025-01-02 to 2025-12-31 (running pipeline.runner.replay quietly if a log is missing). In run(): backtest the configured strategy 2018-2025 with the config cost model and print a metrics table versus SPY buy and hold (total return, CAGR, volatility, Sharpe, Sortino, max drawdown, Calmar, win rate, profit factor, trades, exposure, alpha, beta) and the cost breakdown; backtest 2025 only (starting from 2024-12-31 so the first decision is warm) and compare it with the forward test: total return, Sharpe, max drawdown, orders and average fill slippage in bps versus the day's open; print the tracking difference and annualised tracking error. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Add monitoring checks, the tearsheet and the report (A8, A10)
In labs_fullcode/pipeline/report.py add THRESHOLDS = {'tracking_abs': 0.02, 'slippage_bps': 5.0, 'drawdown_stop': 0.15} and monitoring checks with status OK or ALERT: tracking vs backtest, average fill slippage, drawdown stop rule (current and worst forward-test drawdown), kill switch (pipeline.risk.RiskManager) and diagnostics (read _outputs/Lab18_diagnostics.csv rows for the configured strategy: ALERT on any FAIL or if diagnose was not run for it). Save _outputs/Lab19_monitoring.csv; a 3 x 2 tearsheet Lab19_tearsheet.png (equity vs SPY with the forward-test year shaded, drawdowns, monthly-returns heatmap fixed at +/-8%, rolling 6-month Sharpe, backtest vs forward 2025, costs by type); and _outputs/Lab19_report.md with the tables (DataFrame.to_markdown), five data-supported insights and a recommendation: 'CONTINUE PAPER TRADING ...' if every check is OK, otherwise 'PAUSE AND INVESTIGATE: monitoring alerts on ...'. Paper results only - not investment advice. main(argv) for `python -m pipeline report`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Step 4: Generate the report (A8, A10) (no prompt needed)
Run: python -m pipeline diagnose (so diagnostics exist for ema_cross), then python -m pipeline report (same as: python3 Lab19-performance-report/report_stage.py).
Step 5: Read the monitoring checks (A8) (no prompt needed)
Read section [3]. Which limit is closest to being breached?
Step 6: Trigger an alert on purpose (A8) (no prompt needed)
In pipeline/report.py change THRESHOLDS['tracking_abs'] to 0.005 and rerun python -m pipeline report. Then restore 0.02.
Step 7: Study the tearsheet (A10) (no prompt needed)
Open _outputs/Lab19_tearsheet.png: equity, drawdown, monthly-returns heatmap, rolling Sharpe, backtest vs forward, costs. Find the weakest months.
Step 8: Write the committee recommendation (A10)
Read labs_fullcode/_outputs/Lab19_report.md. Rewrite sections 5 and 6 for Lion City Capital's investment committee in plain English (under 200 words): what the data show, why the forward test matters, the honest message that the strategy reduces risk but has not beaten buy and hold, one additional risk (for example regime change or capacity), and a clear, conditional recommendation. Keep every number unchanged.

Open in Claude | **Open in ChatGPT**

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 20 - Capstone: One Command Runs the Pipeline End to End, with an Agentic Layer and Human Approval

Folder Lab20-agentic-capstone/ - run: python3 Lab20-agentic-capstone/e2e_stage.py

Step 1: Build the agent layer (K1, A7)
Create labs_fullcode/pipeline/agents.py. Agents: technical_analyst(symbol, bars) - EMA 20/50, RSI 14, MACD histogram and ATR 14 from pipeline.indicators -> score in [-1, 1]; news_analyst(cfg, symbol, headlines) - lexicon sentiment of pipeline.ingest.load_headlines (POSITIVE / NEGATIVE word sets); debate(cfg, tech, news) - bull and bear points and balance = 0.6 x technical + 0.4 x news; trader(symbol, tech, research, equity, held) - buy if balance > 0.25 and flat, sell if < -0.25 and held, confidence = min(1, |balance| + 0.3), qty = floor(equity x 0.25 x confidence / close); risk_manager(cfg, proposal, tech, equity) - DETERMINISTIC hard limits from config agents (allow-list, min_confidence, cap = min(equity x max_position_pct, max_order_value) with resizing, ATR bracket stop = close - stop_atr x ATR and target = close + target_atr x ATR). Optional LLM: only if the ANTHROPIC_API_KEY named in secrets_from_env is set and the anthropic package imports, news_analyst and debate call client.messages.create(model=cfg['agents']['model'] (claude-sonnet-5-5), max_tokens=1024, system prompt that treats headlines as untrusted data and asks for ONE JSON object, extra_headers={'anthropic-beta': 'server-side-fallback-2026-07-01'}, extra_body={'fallbacks': 'default'}), return None on refusal or any error so the rule-based brain is used. run(symbols, execute_orders, yes, reset, session) also applies pipeline.risk.RiskManager.check, asks a human to type YES (EOF = declined) before OrderManager.rebalance(..., units='shares', order_type='bracket' in the regular session), writes _outputs/Lab20_decision_trace.json and chart Lab20_agent_pipeline.png. main(argv) for `python -m pipeline agents --symbols --execute --yes --reset --session`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 2: Build the end-to-end orchestrator (A7, A10)
Create labs_fullcode/pipeline/e2e.py. run(execute_orders=False, yes=False, fast=False) calls each stage's run() in order - check (reset), validate, ingest, hypothesis, indicators, reconcile, signals, execute (equal weights over the long universe targets from signals, capped at 20%), evaluate, search, ml, regime, abtest, pairs, backtest, runner.replay for 2025, webhook.selftest, diagnose, report - with contextlib.redirect_stdout into _outputs/Lab20_e2e_log.txt, timing each stage and printing one line: lab, stage, status (OK, GATE or ERROR), seconds and a short result. Exceptions are caught (ERROR) and the run continues. Gates: check not ready, ingest FAIL, diagnose BLOCK, report monitoring ALERT or kill switch on -> agents run propose-only. Last stage: agents.run; with --execute and gates passed ask the human for YES on the real console (or --yes for a scripted demo). Write _outputs/Lab20_e2e_summary.json and chart Lab20_e2e_pipeline.png (one coloured box per stage). --fast skips search, ml and pairs. Exit code 1 if any stage errored. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Step 3: Run the whole pipeline (A7) (no prompt needed)
Run: python -m pipeline e2e (same as: python3 Lab20-agentic-capstone/e2e_stage.py).
Step 4: Trace the run (A10) (no prompt needed)
Open _outputs/Lab20_e2e_log.txt (the full output of every stage), _outputs/Lab20_e2e_summary.json and _outputs/Lab20_e2e_pipeline.png.
Step 5: Read the agents' decisions (K1, A10) (no prompt needed)
Run: python -m pipeline agents --reset and open _outputs/Lab20_decision_trace.json and _outputs/Lab20_agent_pipeline.png. Follow one symbol from evidence to decision.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Run the whole pipeline (A7) (no prompt needed)
Step 6: Apply human oversight (A7, K1) (no prompt needed)
Run: python -m pipeline e2e --execute. When asked, type YES only if you agree with the approved orders; any other answer declines them.
Step 7: Prove the gates stop the orders (A7) (no prompt needed)
Run: python -m pipeline kill --on, then python -m pipeline e2e --fast --execute --yes, then python -m pipeline kill --off.
Step 8: Optional - switch on the LLM agents (K1) (no prompt needed)
Set ANTHROPIC_API_KEY in .env (with the anthropic package installed) and rerun python -m pipeline agents --reset. The news and debate agents then use Claude (claude-sonnet-5-5); if a call fails or is refused the pipeline falls back to the rules. The risk manager and the pipeline risk checks never change.
Step 9: Run the capstone on the live paper account (A7) (no prompt needed)
With Alpaca paper keys set, run python -m pipeline e2e --execute during class. Approved agent orders are sent as extended-hours limit orders (no bracket outside the regular session) - check them in the Alpaca paper dashboard, then flatten with python -m pipeline execute --live-paper --flatten.
Step 10: Evaluate the pipeline (A10, K1)
Using labs_fullcode/_outputs/Lab20_e2e_summary.json and labs_fullcode/_outputs/Lab20_decision_trace.json, draft three sentences for Lion City Capital's investment committee evaluating the end-to-end pipeline: the evidence from the stages, the risks of the agent layer, and the controls that keep it safe. Keep it factual and quote two numbers.

Open in Claude | **Open in ChatGPT**

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.